

Question Bank — Week 10

Advanced sorting (Lecture 10) — Answers

DSA27 · Data Structures and Algorithms

Fall 2026

Contents

Part A — Multiple choice	1
Part B — Short answer and essay	3
Part C — Trace the algorithms	4
Part D — Array state	6
Part E — Complexity analysis	8
Part F — Find and fix the bug	9
Part G — Write the code	10

Questions: [week10-questions.md](#). Commit to your own answer before reading one here. Every trace and count below was produced by running the reference `dsa/sorting.py`, and comparisons were counted with a value class whose `__lt__`, `__le__`, `__gt__` and `__ge__` add to a counter.

Part A — Multiple choice

Q	Ans	Q	Ans	Q	Ans	Q	Ans
M01	d	M07	b	M13	a	M19	b
M02	b	M08	a	M14	b	M20	b
M03	a	M09	b	M15	c	M21	d
M04	c	M10	c	M16	d	M22	a
M05	d	M11	c	M17	a		
M06	c	M12	d	M18	c		

W10-M01 — d. Comparing every pair is what $O(n^2)$ sorts do; divide and conquer is exactly the way to avoid it.

W10-M02 — b. Two sorts of half the size, plus a merge that touches all n . (a) is binary search; (c) is quicksort's worst case; (d) would be a divide and conquer whose combine step is $O(1)$.

W10-M03 — a. Each comparison sends one element to scratch, and the last element needs none: at most $4 + 4 - 1 = 7$, when the two runs interleave to the end (for example $[1, 3, 5, 7]$ and $[2, 4, 6, 8]$).

W10-M04 — c. The $\leq$ in `if a[i] <= a[j]`. With $<$ the sort is still correct but no longer stable (W10-B1). Where it splits (a) has nothing to do with it.

W10-M05 — d. One scratch Array of n slots, shared by every merge, plus an $O(\log n)$ call stack — $O(n)$ in total.

W10-M06 — c. Every merge joins two runs into one; starting from 8 runs of one element, 7 merges leave one run. In general $n - 1$.

W10-M07 — b. On sorted input each merge of two runs of m elements empties the left run after m comparisons: $\frac{n}{2} \log_2 n = 512 \times 10 = 5,120$ (measured). 523,776 (d) is quicksort with `pivot="first"`; 1,023 (a) is Timsort.

W10-M08 — a. The smaller-than-pivot values sit in `a[lo:boundary]`, the others in `a[boundary:j]`; `a[j:hi]` is still unknown, so (d) claims too much.

W10-M09 — b. The pivot is 4; only 3, 1 and 2 are smaller, so it lands at index 3: $[3, 1, 2, 4, 8, 9, 7, 6]$.

W10-M10 — c. One comparison per non-pivot element, $n - 1 = 7$, whatever the order.

W10-M11 — c. Every pivot is the smallest of its range, so each partition peels off one element: $(n - 1) + (n - 2) + \dots + 1$. For $n = 1,024$ that is 523,776 comparisons, measured.

W10-M12 — d. Of the first, middle and last values of a sorted range, the middle one is the median — the exact middle of the range. Median-of-three does cost comparisons (a), and it is not random (b).

W10-M13 — a. Each recursive call is on at most half of its caller's range, so there are at most $\log_2 n$ frames. The **time** is still $O(n^2)$ in the worst case (b). On sorted input with "first" the stack is $O(1)$, but not on every input (d).

W10-M14 — b. A swap in the partition can move an element past its equal, and nothing puts them back. Recursion (a) and in-place work do not by themselves break stability.

W10-M15 — c. With the root at index 0. (b) is the layout with the root at index 1, used in some textbooks.

W10-M16 — d. The sifts at indices 3, 2, 1, 0 give $[4, 10, 3, 7, 1, 8, 2, 5]$, $[4, 10, 8, 7, 1, 3, 2, 5]$, unchanged, and $[10, 7, 8, 5, 1, 3, 2, 4]$ (W10-T4). A heap is not sorted (a, b).

W10-M17 — a. About $n/4$ nodes can sink 1 level, $n/8$ can sink 2, ... — the sum $n(1/4 + 2/8 + 3/16 + \dots)$ is at most n . Measured: about $1.9n$ comparisons, the same ratio from $n = 1,000$ to $8,000$.

W10-M18 — c. From the last parent, $n // 2 - 1 = 4$, down to 0. Indices 5 to 9 are leaves. Going up (d) is W10-B2's bug.

W10-M19 — b. Merge sort (a) needs $O(n)$ extra; quicksort (c) is $O(n^2)$ in the worst case; insertion sort (d) is $O(n^2)$.

W10-M20 — b. The argument counts yes/no answers to comparisons. Counting sort (d) never compares and runs in $O(n + k)$.

W10-M21 — d. Timsort, by Tim Peters (2002): natural runs, insertion sort on short runs, stable merging. Introsort (c) is C++'s `std::sort`.

W10-M22 — a. Timsort finds one ascending run of 1,000 in 999 comparisons and has nothing to merge (measured: 999).

Part B — Short answer and essay

W10-E1 (5)

- **Divide:** split `a[lo:hi]` at `mid = (lo + hi) // 2` — no comparisons. **Conquer:** sort both halves recursively; a range of 0 or 1 elements is sorted (base case). **Combine:** merge the two sorted runs with `_merge`.
- **Recurrence:** $T(n) = 2T(n/2) + cn$, $T(1) = c$.
- **Recursion tree:** the root merges n elements; its two children $n/2$ each; the four grandchildren $n/4$ each — every level totals n . The sizes halve, so there are $\log_2 n$ levels that merge: $T(n) = O(n \log n)$, in every case, because the split never depends on the values.
- **Space:** one scratch Array of n slots, plus $\log_2 n$ stack frames: $O(n)$. Not in place.
- **Stable:** on a tie, `_merge` takes from the **left** run ($\leq$), and the left run holds the elements that came first in the input.

W10-E2 (4)

- Lomuto's partition takes the pivot `a[hi]` and one scan `j = lo..hi-1`. Values smaller than the pivot are swapped to the front, into `a[boundary]`, and `boundary` grows. A final swap puts the pivot at `a[boundary]`; it returns `boundary`.
- **Invariant:** `a[lo:boundary] < pivot <= a[boundary:j]`. True at the start (both empty), kept by each step, and at the end `j = hi`, so every value is on the correct side.
- **Final place:** every value smaller than the pivot is to its left, every other value to its right; sorting either side cannot move any value across, so the pivot's position is its position in the sorted array.
- **No combine:** after both parts are sorted, *left part*, *pivot*, *right part* is already in order — the partition did the combining in advance. Merge sort is the mirror image: free divide, expensive combine.

W10-E3 (4)

- "first" and "last": $O(1)$ to choose. Worst on **sorted** and reverse-sorted input: the pivot is the extreme of its range every time, one element per level, $n(n-1)/2$ comparisons.
- "random": $O(1)$ plus a random number. **No fixed input** is bad for it: the worst case needs bad luck at almost every level, which is vanishingly unlikely; the expected cost is about $1.39 n \log_2 n$.
- "median3": up to three extra comparisons per partition. Exact middle on sorted and reversed input. Its worst case exists — specially built "median-of-three killer" inputs (Musser, 1997) — but it is not an input that occurs by accident.
- **Why sorted input matters:** it is the most common input a real sort sees — data sorted before, with a few changes. A pivot rule that is worst on sorted input is worst exactly when it matters.

W10-E4 (4)

- **Array as tree:** $a[0]$ is the root; the children of $a[i]$ are $a[2i+1]$ and $a[2i+2]$; the parent is $a[(i-1)//2]$. The tree is complete, height $\lfloor \log_2 n \rfloor$; there are no pointers.
- **Build-heap:** sift down every parent from $n//2-1$ down to 0; each sift may assume both subtrees are already heaps. **Sort:** swap the root (the max) with $a[\text{end}]$, shrink the heap to $a[0:\text{end}]$, sift the new root down; repeat for $\text{end} = n-1 \dots 1$. $O(n \log n)$.
- **Build-heap is $O(n)$:** a node at height h sinks at most h levels, and there are at most $n/2^{h+1}$ such nodes: $\sum_h h n/2^{h+1} \leq n$. Most nodes are leaves or near them.
- **Slower in practice:** the sift jumps from i to $2i+1$ — far apart in memory, so caches help little — and it makes more comparisons (measured on random data: about $1.7 n \log_2 n$, against about 1.2 for median-of-three quicksort). Its strength is the guarantee: $O(n \log n)$ worst case with $O(1)$ extra space.

W10-E5 (3)

- **Stable:** elements with equal keys come out in the order they went in.
- **Why:** sorting records by one field after another. Sort students by name, then **stably** by grade: within a grade they stay in name order.
- **In `dsa/sorting.py`:** bubble, insertion and merge sort are stable; selection, quick and heap sort are not.
- **Tuples do not test it:** Python compares (key, tag) tuples by the tag when the keys are equal, so every correct sort — stable or not — puts $(1, "a")$ before $(1, "c")$. A stability test needs values that compare by key **only**, such as a class whose comparison methods look at the key alone.

Part C — Trace the algorithms

W10-T1

i	j	k	compare	copied to scratch[k]
0	4	0	3 > 1	1 (right)
0	5	1	3 ≤ 7	3 (left)
1	5	2	7 ≤ 7	7 from the left
2	5	3	8 > 7	7 (right)
2	6	4	8 ≤ 10	8 (left)
3	6	5	12 > 10	10 (right)

Now $j = 7 = hi$: the right run is empty. The **left** leftover loop copies 12 into `scratch[6]`; the right one does nothing. Six comparisons; the result is [1, 3, 7, 7, 8, 10, 12]. The 7 from the **left** run (index 1) goes first, because $7 \leq 7$ is true: that is stability.

W10-T2

j	a[j]	< 6?	array after	boundary
0	5	yes: swap a[0], a[0]	[5, 8, 1, 9, 3, 7, 2, 6]	1
1	8	no	[5, 8, 1, 9, 3, 7, 2, 6]	1
2	1	yes: swap a[1], a[2]	[5, 1, 8, 9, 3, 7, 2, 6]	2
3	9	no	[5, 1, 8, 9, 3, 7, 2, 6]	2
4	3	yes: swap a[2], a[4]	[5, 1, 3, 9, 8, 7, 2, 6]	3
5	7	no	[5, 1, 3, 9, 8, 7, 2, 6]	3
6	2	yes: swap a[3], a[6]	[5, 1, 3, 2, 8, 7, 9, 6]	4
end		swap a[4], a[7]	[5, 1, 3, 2, 6, 7, 9, 8]	returns 4

Seven comparisons, four swaps plus the final one. The pivot 6 is at index 4: four smaller values to its left, three larger to its right.

W10-T3

#	range	pivot	q	array after
1	0..7	4	3	[3, 1, 2, 4, 8, 9, 7, 6]
2	0..2	2	1	[1, 2, 3, 4, 8, 9, 7, 6]
3	4..7	6	4	[1, 2, 3, 4, 6, 9, 7, 8]
4	5..7	8	6	[1, 2, 3, 4, 6, 7, 8, 9]

After partition 1 the left part 0..2 has 3 elements and the right part 4..7 has 4, so the left — the **smaller** — is a recursive call (partition 2, whose parts 0..0 and 2..2 have one element each and need nothing), and the right is the next turn of the loop (partition 3). There, the left part is empty, so the loop continues with 5..7 (partition 4), whose parts 5..5 and 7..7 are single elements. Four partitions; the reference yields frames highlighting 3, 1, 4 and 6.

W10-T4

step	array
input	[4, 10, 3, 5, 1, 8, 2, 7]
sift index 3 (5; child 7)	[4, 10, 3, 7, 1, 8, 2, 5]
sift index 2 (3; children 8, 2)	[4, 10, 8, 7, 1, 3, 2, 5]
sift index 1 (10; children 7, 1)	[4, 10, 8, 7, 1, 3, 2, 5] — no change
sift index 0 (4 sinks: 10, then 7, then 5)	[10, 7, 8, 5, 1, 3, 2, 4] — the heap
swap a[0], a[7]; sift in a[0:7]	[8, 7, 4, 5, 1, 3, 2, 10]
swap a[0], a[6]; sift in a[0:6]	[7, 5, 4, 2, 1, 3, 8, 10]
swap a[0], a[5]; sift in a[0:5]	[5, 3, 4, 2, 1, 7, 8, 10]

After three steps the three largest, 7, 8 and 10, are in their final places at the back, and $a[0:5]$ is a max-heap of the rest.

W10-T5 — 12, 12, 15 and 12 comparisons (measured).

- **Sorted** [1 ... 8]: in every merge the whole left run is smaller, so it empties after as many comparisons as it has elements, and the right run is copied free: $4 \times 1 + 2 \times 2 + 1 \times 4 = 12$.
- [5, 6, 7, 8, 1, 2, 3, 4]: both halves are sorted inside (4 + 4 comparisons), and in the last merge the **right** run is smaller throughout, so it empties after 4: 12.
- **Reversed**: the mirror image of sorted — in every merge the right run empties first: 12.
- [1, 3, 5, 7, 2, 4, 6, 8]: the last merge alternates between the runs to the very end, $4 + 4 - 1 = 7$ comparisons, on top of 4 + 4: 15.

Merge sort's **moves** are the same for all four; only the comparisons depend on how the runs interleave. The worst case for $n = 8$ is $4 \times 1 + 2 \times 3 + 1 \times 7 = 17$.

Part D — Array state

W10-S1

[6 5 3 1 8 7 2 4]							
[6 5 3 1]				[8 7 2 4]			
[6 5]		[3 1]		[8 7]		[2 4]	
[6]	[5]	[3]	[1]	[8]	[7]	[2]	[4]
(1) [5 6]		(2) [1 3]		(4) [7 8]		(5) [2 4]	
(3) [1 3 5 6]				(6) [2 4 7 8]			
(7) [1 2 3 4 5 6 7 8]							

The merges happen in the order (1) to (7): the left half is sorted completely before the right half is touched. The array after each merge, as `merge_sort_steps` yields it:

merge	range	array
1	a[0:2]	[5, 6, 3, 1, 8, 7, 2, 4]
2	a[2:4]	[5, 6, 1, 3, 8, 7, 2, 4]
3	a[0:4]	[1, 3, 5, 6, 8, 7, 2, 4]
4	a[4:6]	[1, 3, 5, 6, 7, 8, 2, 4]
5	a[6:8]	[1, 3, 5, 6, 7, 8, 2, 4] — already in order
6	a[4:8]	[1, 3, 5, 6, 2, 4, 7, 8]
7	a[0:8]	[1, 2, 3, 4, 5, 6, 7, 8]

14 comparisons in all.

W10-S2

#	range	candidates (first, middle, last)	pivot	array after	then
1	0..8	9, 7, 5	7	[1, 2, 5, 3, 6, 4, 7, 9, 8]	right 7..8 (2) is smaller: recurse; loop on 0..5
2	7..8	9, 9, 8	9 (index 7)	[1, 2, 5, 3, 6, 4, 7, 8, 9]	both parts trivial
3	0..5	1, 5, 4	4	[1, 2, 3, 4, 6, 5, 7, 8, 9]	right 4..5 (2) is smaller: recurse; loop on 0..2
4	4..5	6, 6, 5	6 (index 4)	[1, 2, 3, 4, 5, 6, 7, 8, 9]	both parts trivial
5	0..2	1, 2, 3	2	[1, 2, 3, 4, 5, 6, 7, 8, 9]	both parts one element: done

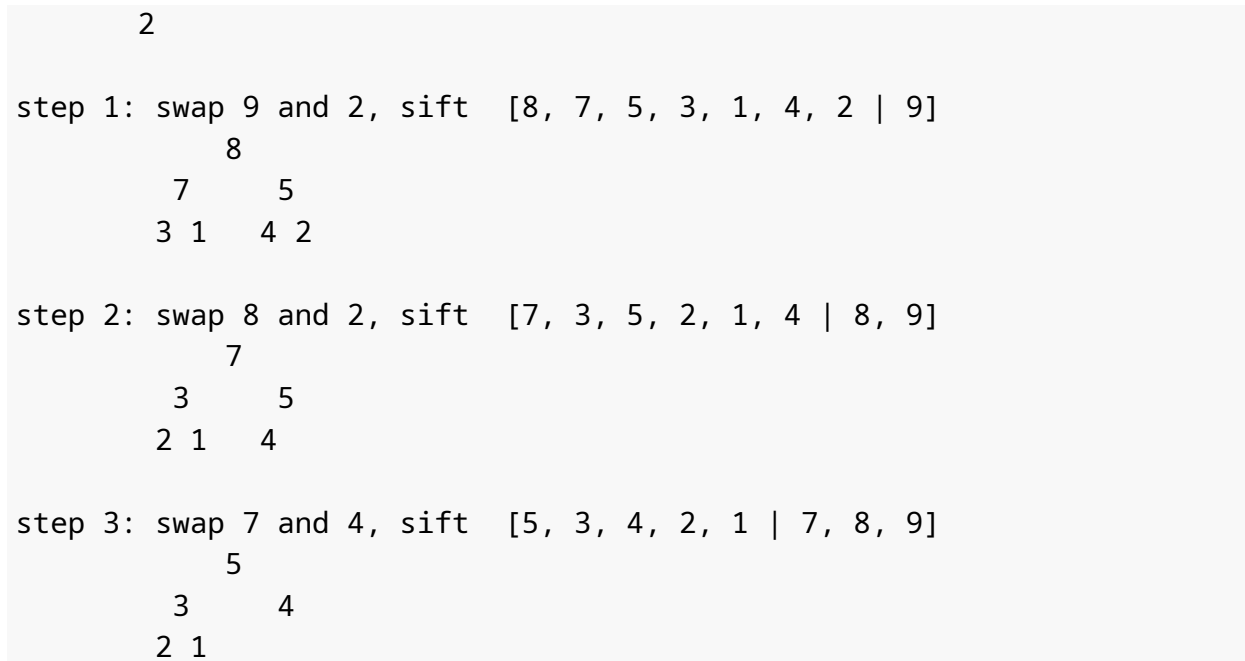
In partition 1, the pivot 7 (index 4) is swapped to the end first, giving [9, 1, 8, 2, 5, 3, 6, 4, 7]; the scan then moves 1, 2, 5, 3, 6 and 4 to the front. In 2 and 4, the first and middle candidates are the same slot, and the tie picks the middle index. The reference yields frames highlighting 6, 8, 3, 5 and 1 — the order above.

W10-S3

after build-heap [9, 7, 8, 3, 1, 4, 5, 2]

```

      9
    7  8
  3 1 4 5
```



Build-heap: index 3 ($7 \geq 3$) and index 1 ($9 \geq 7, 1$) do not move; index 2 swaps 4 with 8; index 0 sinks 2 past 9, 7 and 3. In each sort step the root is swapped with the last slot of the heap (after the bar, sorted and final), and the new root sinks to the larger child each time.

Part E — Complexity analysis

W10-K1

- (a) $\Theta(n \log n)$ — merge sort, every case; quicksort's best case (perfect halves). $\log_2 n$ levels of n work.
- (b) $\Theta(n^2)$: $T(n) = (n-1) + (n-2) + \dots + 1 = n(n-1)/2$. Quicksort's worst case — every pivot the extreme of its range.
- (c) $\Theta(n)$: $n + n/2 + n/4 + \dots < 2n$. Quickselect (W10-C3) when every pivot halves the range: only **one** side is kept, so the work shrinks geometrically instead of staying n per level.

W10-K2 — $T(n) = T(n-1) + (n-1)$, $T(1) = T(0) = 0$: the partition of n elements costs $n-1$ comparisons and leaves an empty left part and $n-1$ elements on the right. So $T(n) = \sum_{m=1}^{n-1} m = n(n-1)/2$ exactly — 4,950 for $n = 100$, 523,776 for $n = 1,024$ (measured). **Stack depth**: with the smaller-side recursion, the smaller part is always the empty left one, so each call returns at once: $O(1)$ extra frames. Without it — two recursive calls — the depth is $n-1$: `list(range(1000))` raises `RecursionError` (W10-B3).

W10-K3 — with $n = 1,000,000$, $\log_2 n \approx 20$:

Method	Cost	$k = 1$	$k = 10$	$k = n/2$
(1) sort, take k	$O(n \log n)$	2×10^7	2×10^7	2×10^7

Method	Cost	k = 1	k = 10	k = n/2
(2) k selection passes	$O(nk)$	10^6	10^7	5×10^{11}
(3) heap, pop k	$O(n + k \log n)$	2×10^6	$\approx 2 \times 10^6$	$\approx 2.2 \times 10^7$

k = 1: one pass (2) — the maximum, $n - 1$ comparisons. **k = 10:** the heap (3) — a tenth of the cost of (2), and ten times cheaper than sorting. **k = n/2:** sorting (1) and the heap are the same order, $O(n \log n)$; the $O(nk)$ method is hopeless. (A fourth way, quickselect for the k-th largest and then sorting the k above it, is $O(n + k \log k)$.)

Part F — Find and fix the bug

W10-B1. < in place of <=: on a tie the element from the **right** run is taken first, so equal keys come out in the wrong order — merge sort is no longer stable. The numbers are still sorted, which is why all 37 tests pass: they sort plain numbers, where two equal values cannot be told apart. With cards compared by key only, [2a, 1b, 2c, 1d] sorts to [1d, 1b, 2c, 2a] instead of [1b, 1d, 2a, 2c] (measured). **Fix:** if `a[i] <= a[j]`:

W10-B2. Going **up** from index 0 sifts a parent before its subtrees are heaps, so a large value deep in the tree cannot reach the root. On [1, 2, 3, 4, 5] ($n = 5$): sifting index 0 swaps 1 with 3 $\rightarrow$ [3, 2, 1, 4, 5]; sifting index 1 swaps 2 with 5 $\rightarrow$ [3, 5, 1, 4, 2]. The root 3 is smaller than its child 5: not a heap. The sort phase then puts 3 at the back as if it were the maximum; the reference with this bug returns [1, 2, 4, 5, 3]. (Only 2 of the 37 tests catch it.) **Fix:** for `index in range(n // 2 - 1, -1, -1)`:

W10-B3. Both parts are recursive calls. On sorted input with `pivot="first"`, every partition leaves an empty left part and all the rest on the right, so the recursion is n deep — one generator frame per element — and `quick_sort(list(range(1000)), pivot="first")` raises `RecursionError: maximum recursion depth exceeded` (measured: 500 still works). The tests do not see it: their worst-case test uses 200 elements and the default "median3". **Fix:** recurse into the smaller part only, and loop on the larger:

```
def sort(lo, hi):
    while lo < hi:
        p = _choose_pivot(a, lo, hi, pivot, rng)
        a[p], a[hi] = a[hi], a[p]
        q = _partition(a, lo, hi)
        yield snapshot(a), (q,)
        if q - lo < hi - q:
            yield from sort(lo, q - 1)
            lo = q + 1
        else:
            yield from sort(q + 1, hi)
            hi = q - 1
```

W10-B4. It returns [5, 2, 9, 1, 7] — unchanged. `sort` is a generator (it contains `yield`), so `sort(lo, mid)` only creates a generator object and throws it away: the halves are never sorted. Only the top-level call runs, and it merges [5, 2] with [9, 1, 7], which are not sorted runs: 5 and 2 are taken first (both below 9), then the rest is copied as it is. **Fix:** `yield` from `sort(lo, mid)` and `yield` from `sort(mid, hi)`.

Part G — Write the code

W10-C1

```
def merge_k_sorted(lists):
    if not lists:
        return []
    runs = [list(run) for run in lists]
    while len(runs) > 1:
        merged = []
        for i in range(0, len(runs) - 1, 2):
            merged.append(_merge_two(runs[i], runs[i + 1]))
        if len(runs) % 2 == 1:
            merged.append(runs[-1])           # the odd one out waits a round
        runs = merged
    return runs[0]

def _merge_two(left, right):
    out = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:               # a tie takes from the left
            out.append(left[i])
            i += 1
        else:
            out.append(right[j])
            j += 1
    out.extend(left[i:])
    out.extend(right[j:])
    return out
```

Why pairs: merging one list after another into a growing result makes the i -th merge cost about the size of everything so far — $O(Nk)$ in all (measured on 64 lists of 64: about 131,000 comparisons). Merging in **pairs**, round after round, is merge sort's recursion tree seen bottom-up: each round touches every value once and halves the number of lists, so there are $\lceil \log_2 k \rceil$ rounds of $O(N)$ — 24,452 comparisons on the same input. Ties take

from the earlier list, so it is stable.

W10-C2

```
def count_inversions(values):
    def sort(items):
        if len(items) <= 1:
            return list(items), 0
        mid = len(items) // 2
        left, inside_left = sort(items[:mid])
        right, inside_right = sort(items[mid:])
        merged, i, j, across = [], 0, 0, 0
        while i < len(left) and j < len(right):
            if left[i] <= right[j]:
                merged.append(left[i])
                i += 1
            else:
                merged.append(right[j])
                j += 1
                across += len(left) - i          # right[j] jumps over all of
        merged.extend(left[i:])
        merged.extend(right[j:])
        return merged, inside_left + inside_right + across

    return sort(list(values))[1]
```

Why `len(left) - i`: when `right[j]` is taken, it is smaller than `left[i]` — and, since `left` is sorted, smaller than every value still waiting in `left[i:]`. All of those came **before** it in the input, so each is one inversion, counted at once. Inversions inside each half are counted by the recursive calls; the merge counts the ones across the middle. $O(n \log n)$; the reversed list of 4,096 values, with 8,386,560 inversions, takes 24,576 comparisons. (The slices copy, but only $O(n)$ per level — the order is unchanged.)

W10-C3

```
def kth_smallest(values, k):
    if not 1 <= k <= len(values):
        raise ValueError("k must be between 1 and len(values)")
    a = list(values)
    rng = random.Random(10)
    lo, hi, target = 0, len(a) - 1, k - 1
    while True:
        p = rng.randint(lo, hi)
        a[p], a[hi] = a[hi], a[p]
        pivot, boundary = a[hi], lo
        for j in range(lo, hi):          # Lomuto, as in quick sort
            if a[j] < pivot:
                a[boundary], a[j] = a[j], a[boundary]
```

```

        boundary += 1
    a[boundary], a[hi] = a[hi], a[boundary]
    if boundary == target:
        return a[boundary]
    if boundary < target:
        lo = boundary + 1           # the answer is on the right
    else:
        hi = boundary - 1         # the answer is on the left

```

Why $O(n)$: after a partition, the answer's position $k - 1$ is on **one** side, and only that side is searched further. With good pivots the work is $n + n/2 + n/4 + \dots < 2n$ (W10-K1 (c)); with random pivots the expected total is at most about $3.4n$ for the median — measured: 11,326 comparisons for $n = 4,096$, about $2.8n$. Quicksort must sort **both** sides, so its levels each cost n . The worst case is still $O(n^2)$, and many equal values push towards it, because the strict $<$ sends every copy of the pivot to one side; a three-way partition (W10-C4) fixes that.

W10-C4

```

def sort_colours(values):
    low, mid, high = 0, 0, len(values) - 1
    while mid <= high:           # values[mid..high] is unknown
        colour = values[mid]
        if colour == 0:
            values[low], values[mid] = values[mid], values[low]
            low += 1
            mid += 1
        elif colour == 1:
            mid += 1
        else:
            values[mid], values[high] = values[high], values[mid]
            high -= 1           # do not advance mid: re-check

```

Invariant: values[0:low] are 0s, values[low:mid] are 1s, values[high+1:] are 2s, and values[mid..high] is still unknown. A 0 is swapped to the end of the 0s — the value it swaps with is a 1 (or itself), so mid may advance; a 2 is swapped to the front of the 2s, and the value that comes back is unknown, so mid stays and it is checked next. Each step shrinks the unknown region by one: one pass, $O(n)$, $O(1)$ extra — at most n swaps. This is Dijkstra's "Dutch national flag" problem, and it is the three-way partition that fixes quicksort on many equal keys.

W10-C5

```

def merge_sort_bottom_up(values):
    a = list(values)
    n = len(a)
    scratch = [None] * n

```

```

width = 1
while width < n:
    for lo in range(0, n, 2 * width):           # merge a[lo:mid] and a[mid:h]
        mid = min(lo + width, n)
        hi = min(lo + 2 * width, n)
        i, j = lo, mid
        for k in range(lo, hi):
            if j >= hi or (i < mid and a[i] <= a[j]):
                scratch[k] = a[i]               # left run: on a tie too
                i += 1
            else:
                scratch[k] = a[j]
                j += 1
        a, scratch = scratch, a                # the merged pass becomes the
    width *= 2
return a

```

Passes: width 1, 2, 4, ... while width < n: $\lceil \log_2 n \rceil$ passes, each reading and writing every element once — $O(n \log n)$, the same recursion tree as Lecture 10, walked from the leaves up instead of from the root down. **Short runs:** mid and hi are capped at n, so the last pair of a pass may have a short right run, or none at all (mid == hi); then the loop simply copies the left run across. For n = 3,000 (12 passes) it made 31,192 comparisons, under $n \lceil \log_2 n \rceil = 36,000$. The condition `j >= hi or (i < mid and a[i] <= a[j])` takes from the left when the right run is used up, and on a tie — stable. Swapping a and scratch after each pass avoids copying back: the merged pass simply becomes the next pass's input. No recursion means no call stack at all.